

Classificazione di immagini mediante deep learning

L.Verdoliva, D.Cozzolino

In questa esercitazione sviluppiamo un approccio basato sulle Reti Neurali Convoluzionali per la classificazione di immagini. A tal fine useremo la libreria *PyTorch* che offre un insieme di strumenti flessibili per: la preparazione dei dati, la definizione dell'architettura della rete neurale, e l'addestramento. PyTorch supporta l'esecuzione su GPU Nvidia, riducendo enormemente i tempi di calcolo. Per questa esercitazione potete utilizzare il servizio Google Colaboratory (o Colab) oppure installare PyTorch sul vostro computer, eseguendo nel Prompt di Anaconda:

```
conda activate course
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

Colab permette di eseguire codice Python in un qualsiasi browser web, inoltre non richiede una configurazione in quanto già risultano installate le principali librerie Python incluse le librerie per il deep learning come PyTorch. Per utilizzare Colab, avete bisogno di un account Google e di attivare il servizio alla pagina:

<https://gsuite.google.com/u/1/marketplace/search/Colaboratory>. Una volta attivato il servizio, potete creare un nuovo Notebook Colab all'interno di una cartella di Google Drive. Al tal fine, accedete tramite browser a Google Drive (<https://drive.google.com/drive/my-drive>), create una cartella per il corso in cui salverete i vari Notebook, e selezionate "Google Colaboratory" dal menu "New" per creare il vostro primo Notebook Colab. Un Notebook Python rispetto a uno script è organizzato in celle. Ogni cella può essere eseguita singolarmente e il relativo output apparirà al di sotto della cella nella stessa pagina web.

1 Classificazione usando LeNet

Come primo esempio di classificazione affrontiamo il problema del riconoscimento delle cifre da 0 a 9 scritte a mano e usiamo il dataset MNIST composto da un totale di 70.000 immagini su scala di grigi di 28×28 pixel. Alcune immagini di esempio sono riportate in figura 1. Per comodità dividiamo il codice da scrivere in quattro sezioni:

1. preparazione dei dati;
2. definizione dell'architettura;
3. addestramento;
4. analisi delle prestazioni.

Prima di iniziare, resettiamo la console e importiamo le librerie che useremo:

```
%reset -f
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data
```

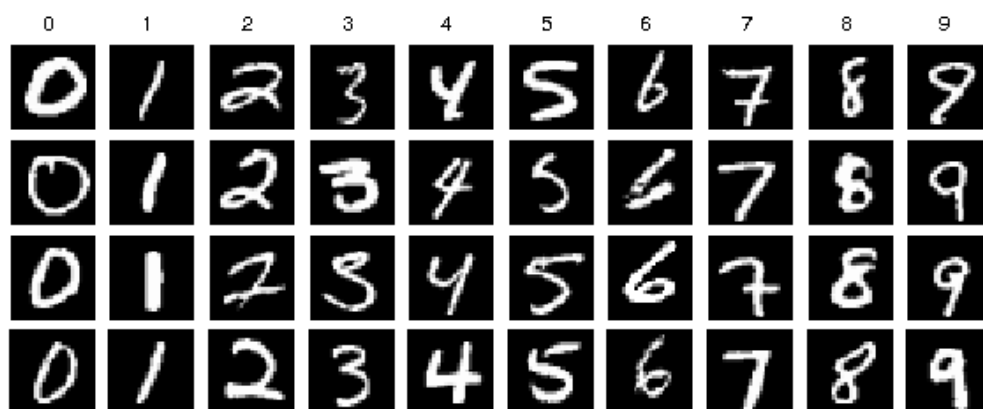


Figura 1: Esempi di immagini del dataset MNIST.

1.1 Preparazione dei dati

In PyTorch, per ogni set di dati si definiscono due oggetti Python:

1. `torch.utils.data.Dataset`: che si occupa del caricamento dei dati e applica eventuali trasformazioni;
2. `torch.utils.data.DataLoader`: che si occupa del raggruppamento in batch e del mescolamento casuale (shuffling).

La libreria `torchvision` fornisce già un'implementazione dei Dataset per il dataset MNIST, che possiamo istanziare nel seguente modo:

```
from torchvision.datasets import MNIST
from torchvision.transforms.functional import to_tensor

trainval_set = MNIST(train=True, download=True, root='./data/', transform=to_tensor)
test_set = MNIST(train=False, download=True, root='./data/', transform=to_tensor)

print('numero immagini training e validation set:', len(trainval_set))
print('numero immagini test set:', len(test_set))
```

In questo modo, abbiamo due oggetti Dataset uno per il training/validation set (60.000 immagini) ed uno per test set (10.000 immagini). Notate che abbiamo specificato al parametro `transform`, la funzione `to_tensor` della libreria `torchvision`. Questa funzione converte le immagini in tensori float32 nel range `[0,1]` e riordina gli assi dalla convenzione Channels-Last (altezza $\times$ larghezza $\times$ canali) alla convenzione Channels-First (canali $\times$ altezza $\times$ larghezza) che è quella usata da PyTorch.

Separiamo le 60.000 immagini in un validation set (5.000 immagini) e un training set (55.000 immagini) usando la funzione `random_split`:

```
train_set, val_set = data.random_split(trainval_set, [55000, 5000])
print(f'Training set: {len(train_set)} immagini')
print(f'Validation set: {len(val_set)} immagini')
print(f'Test set: {len(test_set)} immagini')
```

Visualizziamo alcune immagini del test set:

```
# Visualizzazione le prima 10 immagini del test set
plt.figure()
for index in range(10):
    img, label = test_set[index]
    # img è un tensore float32 con shape (1,28,28), label è un intero da 0 a 9
    img = img[0].cpu().numpy() # conversione da torch (1,28,28) a numpy (28,28)
    plt.subplot(2, 5, index+1)
    plt.imshow(img, cmap='gray', clim=[0, 1])
    plt.title(f'Etichetta: {label}')
    plt.axis('off')
plt.suptitle('Esempi dal test set MNIST')
plt.tight_layout() # riduce spazio bianco tra i subplot
plt.show()
```

Creiamo i DataLoader per ciascun set. Il parametro `batch_size` definisce il numero di immagini per ogni batch, mentre `shuffle=True` mescola il training set ad ogni epoca:

```
train_loader = data.DataLoader(train_set, batch_size=200, shuffle=True)
val_loader   = data.DataLoader(val_set,   batch_size=200, shuffle=False)
test_loader  = data.DataLoader(test_set,  batch_size=200, shuffle=False)
```

1.2 Definizione dell'architettura della rete neurale

Implementiamo l'architettura LeNet (figura 2) la cui struttura è descritta nella Tabella 1. In PyTorch, le architetture sono oggetti Python che ereditano da `torch.nn.Module`. In questo caso l'architettura è una sequenza di layer standard, ed useremo `torch.nn.Sequential` per definirla.

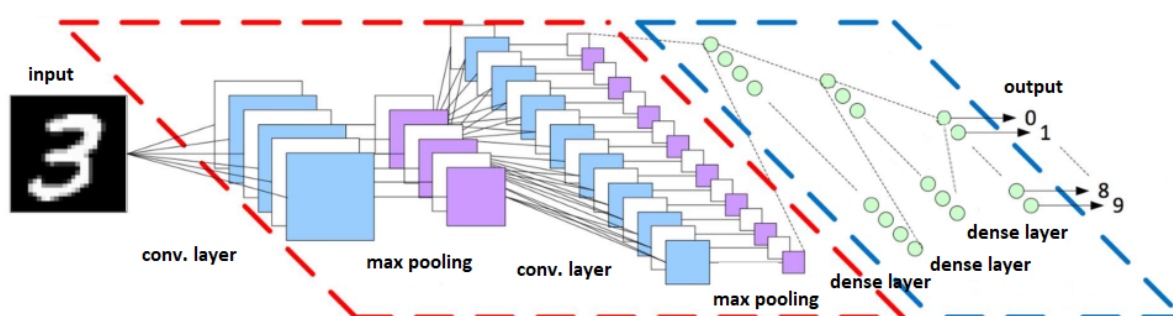


Figura 2: Architettura LeNet.

	Tipo	Supporto Spaziale	Numero di Filtri	Attivazione
1	Convoluzionale	5×5	6	ReLU
2	Max Pooling	2×2	-	-
3	Convoluzionale	5×5	16	ReLU
4	Max Pooling	2×2	-	-
5	Fully Connected	-	120	ReLU
6	Fully Connected	-	84	ReLU
7	Fully Connected	-	10	Softmax

Tabella 1: Elenco degli strati dell'architettura LeNet.

```

lenet = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2),    # output: 6 x 28 x 28
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),        # output: 6 x 14 x 14
    nn.Conv2d(6, 16, kernel_size=5, padding=0),  # output: 16 x 10 x 10
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),        # output: 16 x 5 x 5
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120),
    nn.ReLU(),
    nn.Linear(120, 84),
    nn.ReLU(),
    nn.Linear(84, 10),
    nn.LogSoftmax(), # restituiamo i logaritmo delle probabilità (logit)
)

print(lenet) # Stampa del sommario dell'architettura

```

Abbiamo usato il metodo `nn.Conv2d` per creare gli strati convoluzionali, specificando nell'ordine: il numero di canali in ingresso, il numero di filtri, la dimensione dei filtri e il numero di pixel di zero-padding. Con il metodo `nn.ReLU` abbiamo invece creato gli strati che applicano la funzione di attivazione non lineare. Con `nn.MaxPool2d` creiamo gli strati che eseguono il max pooling, indicando la dimensione della finestra e lo spostamento (*stride*) tra le finestre. Lo strato definito con `nn.Flatten` serve a convertire l'immagine in un vettore: si passa da un array di dimensioni $5 \times 5 \times 16$ a un vettore di 400 elementi. Gli strati fully connected sono creati con il metodo `nn.Linear`, specificando le dimensioni di input e di output. Infine, abbiamo creato uno strato che calcola il logaritmo della softmax, in modo che la rete restituisca i logaritmi delle probabilità associate alle 10 classi.

1.3 Addestramento

In PyTorch la fase di addestramento va scritta esplicitamente. Prima definiamo la loss function, l'ottimizzatore e il numero di epoche:

```

loss_function = nn.CrossEntropyLoss() # accetterà logit e etichette intere
optimizer = optim.SGD(lenet.parameters(), lr=0.01) # ottimizzatore
num_epochs = 10 # numero di epoche

```

Scriviamo ora il ciclo di addestramento. Iteriamo sul numero di epoche, ed in ogni ogni epoca iteriamo sui batch del training set usando `train_loader`. Per ogni batch di immagini: (1) azzeriamo i gradienti, (2) applichiamo la rete, (3) calcoliamo il valore di loss, (4) calcoliamo i gradienti e (5) aggiorniamo i parametri.

```
for epoch in range(num_epochs):
    # ---- Fase di TRAINING ----
    lenet.train() # abilita comportamenti specifici del training (es. dropout)
    cum_loss = 0.0
    num_images = 0

    for images, labels in train_loader:
        optimizer.zero_grad()           # (1) azzeri i gradienti
        logits = lenet(images)           # (2) applica la rete
        loss = loss_function(logits, labels) # (3) calcolo del valore di loss
        loss.backward()                  # (4) calcolo gradienti
        optimizer.step()                 # (5) aggiornamento dei pesi

        # Aggiornamento delle statistiche
        cum_loss += loss.item() * images.shape[0]
        num_images += images.shape[0]

    train_loss = cum_loss / num_images
    print(f'Epoca [{epoch}/{num_epochs}] Loss train: {train_loss:.4f}')
```

Ricordiamoci che ad fine di ogni epoca dobbiamo valutare le prestazioni sul validation set:

```
for epoch in range(num_epochs):
    # ---- Fase di TRAINING ----
    ...
    # ---- Fase di VALIDAZIONE ----
    lenet.eval() # disabilita comportamenti specifici del training
    val_cum_loss = 0.0
    val_cum_correct = 0
    val_num_images = 0

    with torch.no_grad(): # disabilita il calcolo dei gradienti
        for images, labels in val_loader:
            logits = lenet(images)
            loss = loss_function(logits, labels)
            predicted = torch.argmax(logits, -1)
            val_cum_loss += loss.item() * images.shape[0]
            val_cum_correct += torch.sum(predicted==labels).item()
            val_num_images += images.shape[0]

    val_loss = val_cum_loss / val_num_images
    val_acc = val_cum_correct / val_num_images

    print(f'Epoca [{epoch}/{num_epochs}] ',
          f'Loss val: {val_loss:.4f} Acc val: {val_acc:.4f}')
```

1.4 Analisi delle prestazioni

Prima di valutare le prestazioni globali, testiamo la rete su una singola immagine:

```
# Test su una singola immagine
ID = 1000 # indice dell'immagine nel test set

img, label = test_dataset[ID] # img: tensore (1, 28, 28)
batch_input = img.unsqueeze(0) # aggiungiamo la dimensione batch ->(1, 1, 28, 28)

lenet.eval()
with torch.no_grad():
    logits = lenet(batch_input)
    probs = torch.softmax(logits, 1) # probabilità per ogni classe
    pred = torch.argmax(logits, 1).item() # classe predetta

probs = probs[0].cpu().numpy() # conversione da torch (1, 10) a numpy (10,)
img = img[0].cpu().numpy() # conversione da torch (1, 28, 28) a numpy (28, 28)

plt.figure()
plt.subplot(1,2,1); plt.imshow(img, cmap='gray', clim=[0, 1])
plt.title(f'Etichetta vera: {label} \n Etichetta predetta: {pred}')
plt.subplot(1,2,2); plt.bar(range(10), probs)
plt.xticks(range(10)); plt.grid() # visualizza la griglia
plt.show()
```

Per valutare le prestazioni sull'intero test set:

```
# Valutazione sull'intero test set
from sklearn.metrics import balanced_accuracy_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

lenet.eval()
y_true = []
y_pred = []

with torch.no_grad():
    for images, labels in test_loader:
        logits = lenet(images)
        predicted = torch.argmax(logits, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

bAcc = balanced_accuracy_score(y_true, y_pred)
cm = confusion_matrix(y_true, y_pred)

ConfusionMatrixDisplay(cm).plot() # visualizza la matrice di confusione
plt.title(f'bAcc:{bAcc}')
plt.show()
```

1.5 Esecuzione su GPU

Per abilitare la GPU su Colab, accedete a "Modifica > Impostazioni notebook" e selezionate GPU come acceleratore hardware. Prima dell'addestramento, spostiamo la rete sulla GPU:

```
device = 'cuda:0' # identificativo della GPU Nvidia
lenet = lenet.to(device)
```

Ad ogni iterazione, spostiamo le immagini e le etichette sulla GPU:

```
images = images.to(device)
labels = labels.to(device)
```

1.6 Esercizi proposti

1. *Learning-Rate*. Ripetete l'esercitazione cambiando il learning rate. Verificate il comportamento con un learning rate alto ($lr=1.0$) e uno basso ($lr=0.0001$).
2. *Dataset CIFAR-10*. Utilizzate LeNet per la classificazione di oggetti usando il dataset CIFAR-10 composto da 60.000 immagini 32×32 a colori di 10 classi. Attenzione: le immagini CIFAR-10 hanno 3 canali (RGB), quindi bisogna modificare il primo layer della rete. Caricate i Dataset usando la funzione `torchvision.datasets.CIFAR10`